

Programación de Sistemas de Telecomunicación

Ejercicio 40 - Proyecto Final

GroupChat v2

GIT – GITT – GIST
GSyC, EIF, URJC

Diciembre de 2024

Resumen

El objetivo de este proyecto es implementar en Rust un servicio de chat grupal basado en el modelo cliente–servidor sobre TCP.

Cada usuario (humano) del GroupChat ejecutará un cliente, que se conectará por TCP al servidor. Una vez conectado, el cliente podrá unirse a un grupo, enviar mensajes al resto de clientes unidos a ese grupo, recibir los mensajes que otros clientes del grupo envíen, y abandonar un grupo.

Todos los clientes del GroupChat se conectan al mismo servidor, que gestiona los clientes conectados y los grupos en los que se encuentran. El servidor recibe los mensajes de cada cliente y los reenvía a los otros clientes que están en su mismo grupo.

El formato de los mensajes entre cliente y servidor que se especifica debe ser respetado estrictamente, y de esta forma será posible que puedan interactuar clientes y servidores desarrollados por diferentes estudiantes.

1. Especificación del cliente

1.1. Argumentos en la línea de comandos del cliente

El cliente recibirá obligatoriamente 3 argumentos en la línea de comandos cuando se arranca: `ip-servidor`, `puerto-servidor`, `nickname`

1.2. Interfaz de usuario del cliente

Una vez arrancado, el cliente leerá líneas de la entrada estándar con comandos introducidos por el usuario. Los comandos que deberá reconocer son los siguientes:

- `.list`
Solicita al servidor la lista de grupos que existen actualmente
- `.create <grupo>`
Crea un grupo en el servidor.
- `.join <grupo>`
Se une a un grupo. El resto de clientes de ese grupo recibirán un mensaje indicando que este *nickname* se ha unido al grupo
- `.leave`
Se abandona el grupo en el que está. El resto de clientes de ese grupo recibirán un mensaje indicando que este *nickname* ha abandonado el grupo.
- `.quit`
Se desconecta del servidor y termina la ejecución.

- `texto-de-mensaje-para-grupo`

Cualquier línea introducida por el usuario que no empiece por `.` se entenderá que es un mensaje para enviar a los miembros del grupo en el que está actualmente. El resto de clientes que estén unidos a ese grupo verán en su terminal un mensaje con formato `nickname: texto-de-mensaje-para-grupo`.

Si no se está unido a ningún grupo, se le informará de esta circunstancia al usuario sin enviar ningún mensaje al servidor.

1.3. Comportamiento del cliente

El cliente al arrancar abrirá la conexión TCP con el servidor especificado por su IP y puerto en la línea de comandos. Tras establecer la conexión, enviará un mensaje **NEWUSER** conteniendo su *nickname* (ver los formatos de todos los mensajes en 3). Como respuesta puede recibir uno de estos dos mensajes:

- **ACCEPT**, que indica que su *nickname* ha sido aceptado y ha entrado en el GroupChat. El mensaje **ACCEPT** incluye un *hash* que el cliente tendrá que enviar en el resto de los mensajes. El cliente guardará el *nickname* y el *hash* en un fichero. Si al arrancar el cliente tiene ese fichero y va a utilizar el mismo *nickname*, el cliente NO enviará el mensaje **NEWUSER** y utilizará directamente el *hash* del fichero en sus mensajes.
- **REJECT**, que indica que su *nickname* ya estaba en uso en el GroupChat. En este caso el cliente cerrará la conexión y terminará su ejecución.

Una vez el cliente es aceptado en el servidor, puede utilizar los comandos especificados en 1.2:

- El comando `.list` hará que el cliente envíe el mensaje **LIST**. Como respuesta recibirá del servidor un mensaje **INFO** cuyo contenido es la lista de grupos.
- El comando `.create` hará que el cliente envíe el mensaje **CREATE**. Como respuesta recibirá del servidor un mensaje **INFO** cuyo contenido indicará si se ha creado el grupo, o si no se ha podido crear porque ya existía un grupo con ese nombre.
- El comando `.join` hará que el cliente envíe el mensaje **JOIN**. Como respuesta recibirá del servidor un mensaje **INFO** cuyo contenido indicará si se ha unido al grupo, si no se ha unido porque el nombre de grupo indicado no existiera, o no se ha unido porque aún está dentro de un grupo que debería abandonar primero.
- El comando `.leave` hará que el cliente envíe el mensaje **LEAVE**. Como respuesta recibirá del servidor un mensaje **INFO** cuyo contenido indicará si ha abandonado el grupo, o si no ha abandonado el grupo porque actualmente no estaba en ninguno.
- El comando `texto-de-mensaje-para-grupo` hará que el cliente envíe un mensaje **TEXT**. Si el usuario está unido a un grupo, no recibirá respuesta a este mensaje. Si el usuario no está unido a ningún grupo, recibirá como respuesta un mensaje **INFO** cuyo contenido indicará que actualmente no pertenece a ningún grupo.
- El comando `.quit` hará que el cliente envíe un mensaje **QUIT**. Como respuesta no se recibirá ningún mensaje y se terminará la ejecución del cliente.

Una vez que el cliente se haya unido a un grupo, recibirá mensajes **INFO** como consecuencia de los mensajes que envían otros usuarios unidos al mismo grupo, incluyendo mensajes que indican alguno de ellos abandona el grupo o bien algún nuevo usuario se une a él.

1.4. Pautas de implementación del cliente

1. Al empezar el programa, el cliente enviará el **NEWUSER** (salvo que ya tenga un *hash* para su *nickname*) y recibirá su respuesta. Si la respuesta es **ACCEPT**, el programa principal entrará en un bucle leyendo líneas de `stdin` y enviando mensajes al servidor. Además, se creará un nuevo *thread* que será el encargado de leer líneas del `TcpStream` de la conexión que contendrán los mensajes **INFO** que le envíe el servidor y mostrarlos en la salida estándar. Antes de lanzar el *thread* habrá que clonar el `TcpStream` con `try_clone()` para que el *thread* utilice el *stream* clonado.
2. Es aconsejable que el programa principal escriba en el `TcpStream` directamente con `writeln!()`, y que el *thread* lea del *stream* clonado recubriéndolo primero con un `BufReader` y utilizando su método `lines()`.
3. Para que se diferencien visualmente en la salida las líneas que escribe el *thread* de las líneas con los comandos que escribe el usuario es aconsejable emplear colores utilizando el `crate colored`, como en el programa `show_packets`.

2. Especificación del servidor

2.1. Argumentos en la línea de comandos del servidor

El servidor recibirá obligatoriamente 1 argumento en la línea de comandos cuando se arranca: `puerto-servidor`.

2.2. Interfaz de usuario del servidor

El servidor irá mostrando en la salida estándar mensajes que informen sobre lo que está haciendo. El formato de esta información es libre, pero deberá servir para comprobar el funcionamiento del servidor.

2.3. Comportamiento del servidor

El servidor esperará conexiones en todas sus interfaces de red y en el puerto especificado en la línea de comandos.

Para cada conexión procedente de un cliente, lanzará un *thread* que se encargue de recibir los mensajes procedentes de este cliente.

El servidor recibirá de un mismo cliente diversos mensajes. Si el mensaje recibido es **LIST**, **CREATE**, **JOIN**, **LEAVE**, **TEXT** o **QUIT** (ver los formatos de todos los mensajes en 3), el mensaje debe contener un *hash* que el servidor conozca. En caso contrario, el servidor ignorará ese mensaje sin enviar ninguna respuesta al cliente.

Si el mensaje recibido por el servidor es **NEWUSER**, comprobará si *nickname* está ya en uso en el servidor, en cuyo caso responderá con un mensaje **REJECT**. Si el *nickname* es nuevo, responderá con un mensaje **ACCEPT**. Este mensaje **ACCEPT** incluirá un *hash* generado a partir del *nickname*. Este *hash* lo incluirá el cliente en todos sus siguientes mensajes, como se describe en 3.

El objetivo del *hash* es que otro cliente no pueda «quitar» un *nickname* a un usuario ya existente, o pueda enviar mensajes haciéndose pasar por él.

Si un cliente nunca ha estado conectado al servidor, o bien está utilizando un *nickname* que no ha utilizado anteriormente, su primer mensaje al servidor será **NEWUSER**. Si un cliente se vuelve a conectar al servidor utilizando el *nickname* que ya utilizó anteriormente, empezará a enviar directamente otros mensajes incluyendo el *hash* que obtuvo anteriormente, sin enviar el mensaje **NEWUSER**. Por lo tanto, el servidor debe recordar todos los *nicknames* y *hashes* desde que comenzó su ejecución.

Para cada mensaje recibido con un *hash* es válido, el servidor deberá actuar en la siguiente forma en función del tipo de mensaje:

- **LIST**: El servidor responderá al cliente con un mensaje **INFO** que incluya la lista de grupos actualmente existente en el servidor.
- **CREATE**: El servidor responderá al cliente con un mensaje **INFO** informándole que se ha creado el grupo, o en caso de que ese grupo ya existiera, indicándole esta situación.
- **JOIN**: El servidor comprobará que el cliente no está actualmente en un grupo y que el grupo solicitado existe. En ese caso, le incluirá en el grupo, respondiéndole con un mensaje **INFO** informándole de que se ha unido al grupo, y enviará también al resto de usuarios unidos a ese grupo un mensaje **INFO** informando de que ese *nickname* ha entrado en el grupo. En caso de que el cliente ya estuviera en un grupo antes del **JOIN**, o en caso de que el grupo al que quiere unirse no existiera, el servidor responderá al cliente con un mensaje **INFO** informándole de que no puede unirse al grupo.
- **LEAVE**: El servidor comprobará que el cliente está actualmente en un grupo. En ese caso, le sacará del grupo, respondiéndole con un mensaje **INFO** informándole de que ha abandonado el grupo, y enviará también al resto de usuarios unidos a ese grupo un mensaje **INFO** informando de que ese *nickname* ha abandonado el grupo. Si el cliente no estuviera unido a ningún grupo antes del **JOIN**, el servidor responderá al cliente con un mensaje **INFO** informándole de que no puede salir de ningún grupo.
- **TEXT**: Si el cliente está unido a un grupo, el servidor enviará un mensaje **INFO al resto de usuarios del grupo (sin contar al cliente que envió el mensaje)**, conteniendo el *nickname* del cliente y texto del mensaje. Si el cliente no está unido a ningún grupo antes del mensaje **TEXT**, el servidor el enviará un mensaje **INFO** informándole de que aún no está unido a ningún grupo.
- **QUIT**: Si el cliente está unido a un grupo, envía un mensaje al resto de usuarios del grupo informándoles de que ese *nickname* se ha desconectado del chat. Así mismo, eliminará al cliente de ese grupo y toda la información asociada a él excepto la asociación de su *nickname* y *hash*. Tras esto, terminará la ejecución del *thread* que atiende a este cliente lo que cerrará la conexión con él.

2.4. Pautas de implementación del servidor

1. El servidor utilizará un *thread* para atender a cada cliente de forma concurrente con el resto de clientes.
2. Es aconsejable que el *thread* de cada conexión clone el `TcpStream` para recubrir el clonado con un `BufReader`, y lea de él con el método `readline()`. Y escriba utilizando directamente el `TcpStream` con `writeln!()`.
3. El servidor mantendrá las estructuras de datos necesarias para poder gestionar los grupos y los usuarios del chat. Como los datos estarán compartidos entre los diversos *threads* del servidor, deberán utilizarse mediante `Arc<Mutex<T>>`. No se fija exactamente qué tipo de estructuras de datos deben utilizarse, quedando a la elección de cada estudiante.
4. El *thread* que se encarga de atender la conexión de un cliente recibe/envía mensajes de/para ese cliente a través del *stream* de la conexión. Pero cuando ese *thread* tiene que enviar un mensaje **INFO** a **otros** clientes, necesita tener acceso a los *streams* de las conexiones de los otros clientes. Por lo tanto, entre las estructuras de datos compartidas entre *threads*, tiene que haber alguna que tenga asociado cada cliente con su stream. Para ello, esa estructura tendrá que utilizarse también a través de un `Arc<Mutex<T>>`.
5. Para el cálculo del *hash* asociado a cada cliente, podrá utilizarse la función `hash_string()` del ejemplo siguiente:

```
use std::collections::hash_map::DefaultHasher;
use std::hash::{Hash, Hasher};

fn hash_string(input: &str) -> u64 {
5     let mut hasher = DefaultHasher::new();
    input.hash(&mut hasher);
    hasher.finish()
}

10 fn main() {
    let my_string = "Hola, Rust!";
    let hashed_value = hash_string(my_string);
    println!("El hash de '{}' es: {}", my_string, hashed_value);
}
```

El valor del *hash* debe calcularse concatenando el *nickname* del usuario con la conversión a *string* de un valor entero al azar. Para generar un entero al azar entre *a* y *b* puede utilizarse el *crate* `rand` y su función `rand::thread_rng().gen_range(a..=b)`.

3. Formato de los mensajes

3.1. Mensajes enviados por un cliente al servidor

3.1.1. Mensaje **NEWUSER**

- Formato: `.newuser nickname`
- Significado: Es el primer mensaje que debe enviar un cliente al servidor, con el *nickname* que utilizará en el chat (que el cliente obtiene de su línea de comandos).

3.1.2. Mensaje **LIST**

- Formato: `hash .list`
- Significado: Pide la lista de grupos que existen actualmente en el servidor. Contiene el *hash* que el cliente recibió en el mensaje **ACCEPT** o bien recuperó de un fichero de una ejecución anterior.

3.1.3. Mensaje **CREATE**

- Formato: `hash .create grupo`
- Significado: Pide al servidor la creación de un grupo. Contiene el *hash* que el cliente recibió en el mensaje **ACCEPT** o bien recuperó de un fichero de una ejecución anterior.

3.1.4. Mensaje **JOIN**

- Formato: `hash .join grupo`
- Significado: Pide al servidor entrar en el grupo de nombre `grupo`. Contiene el *hash* que el cliente recibió en el mensaje **ACCEPT** o bien recuperó de un fichero de una ejecución anterior.

3.1.5. Mensaje **LEAVE**

- Formato: `hash .leave`
- Significado: Pide al servidor salir del grupo en el que está actualmente. Contiene el *hash* que el cliente recibió en el mensaje **ACCEPT** o bien recuperó de un fichero de una ejecución anterior.

3.1.6. Mensaje **TEXT**

- Formato: `hash texto`
- Significado: Envía `texto` al servidor para que éste se lo haya llegar a los usuarios del grupo al que pertenece actualmente el cliente. Contiene el *hash* que el cliente recibió en el mensaje **ACCEPT** o bien recuperó de un fichero de una ejecución anterior.

3.1.7. Mensaje **QUIT**

- Formato: `hash .quit`
- Significado: Indica al servidor que se va a desconectar del chat. Contiene el *hash* que el cliente recibió en el mensaje **ACCEPT** o bien recuperó de un fichero de una ejecución anterior.

3.2. Mensajes enviados por el servidor a un cliente

3.2.1. Mensaje **ACCEPT**

- Formato: `.accept nickname hash`
- Significado: El servidor acepta la entrada del cliente en el chat con el nick `nickname`. El mensaje incluye el `hash` calculado por el servidor para ese cliente, que el cliente debe incluir en todos sus mensajes.

3.2.2. Mensaje **REJECT**

- Formato: `.reject nickname`
- Significado: El servidor rechaza la entrada del cliente en el chat con el nick `nickname`.

3.2.3. Mensaje **INFO**

- Formato: `.info texto`
- Significado: El servidor informa al cliente de algo. El contenido de `texto` puede ser uno de los siguientes:
 - `No puedes unirte al char: el nick ya está en uso`
 - `Lista de grupos: grupo_1, grupo_2`
 - `Grupo grupo_1 creado`
 - `Ese grupo ya está creado`

- Te has unido a grupo_1
- nick se ha unido al grupo
- Primero tienes que salirte de tu grupo actual
- grupo_1: grupo inexistente
- Te has salido de grupo_1
- nick ha salido del grupo
- Aún no estás en ningún grupo
- nick: hola a todos
- nick se ha desconectado del chat

NOTA: El contenido exacto del `texto` de los mensajes **INFO** se da a título orientativo.

4. Entrega

El proyecto final se entregará a través de una tarea en el aula virtual, **con límite el 21 de enero a las 09:00h.**

Deben entregarse los siguientes ficheros:

- El código fuente del proyecto `cargo` en único fichero comprimido `.tgz` creado desde el directorio padre del que contiene el proyecto, y que NO incluya el directorio `/target`. Por ejemplo, si el proyecto `cargo` se llama `groupchat`, el fichero comprimido se creará con el comando:

```
tar -cvzf groupchat.tgz --exclude=groupchat/target groupchat
```

Si el código fuente está distribuido en varios proyectos `cargo` (por ejemplo, uno para el cliente y otro para el servidor), se entregará cada uno de ellos comprimido en un fichero `.tgz`.

- Un vídeo tipo *screencast* en el que se muestre la ejecución del programa, lanzando el servidor y varios clientes que utilizan el chat.
- Un vídeo tipo *screencast* en el que el estudiante muestra su código fuente mientras lo explica.